

2025 Jan Oppe 1 - Set 2

Section 2 - problem 2

Most Frequent Numbers form the input

PROBLEM STATEMENT

Write a program to find the most frequent numbers from the given numbers in the input. The most frequent values should be printed in ascending order.

Input Format: The first line contains an integer n . The next n lines each contain n numbers.



EXAMPLE & EXPLANATION

Given the input:
10 3 5 3 2 5 8 8 0

Explanation:

- The number of inputs is 10. The 10 numbers are 3, 5, 3, 2, 5, 8, 3, 8, 5, 0.
- The numbers 3 and 5 both occur three times, which is the maximum frequency.
- The output is a list containing these most frequent numbers sorted in ascending order.

Output: [3, 5]

The Problem Specification

INPUT

`n (int)`
`n numbers`
`(int)`

Integer inputs.

THE BLACK BOX

1. Find the most frequent numbers.
2. Sort them in ascending order.

OUTPUT

`(list of numbers)`

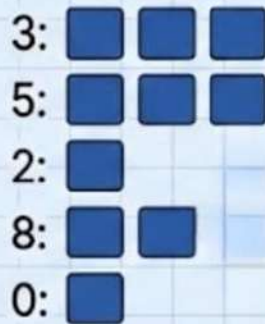
Single List Return Structure.

Constraint: The output must be returned as a single data structure (tuple), not two separate returns.

Visualizing Most Frequent Numbers

Example Data: $n = 10$, List = `[3, 5, 3, 2, 5, 8, 3, 8, 5, 0]`

Step 1: Count Frequencies



Maximum frequency is 3.
Numbers: 3 and 5.

Step 2: Filter and Sort

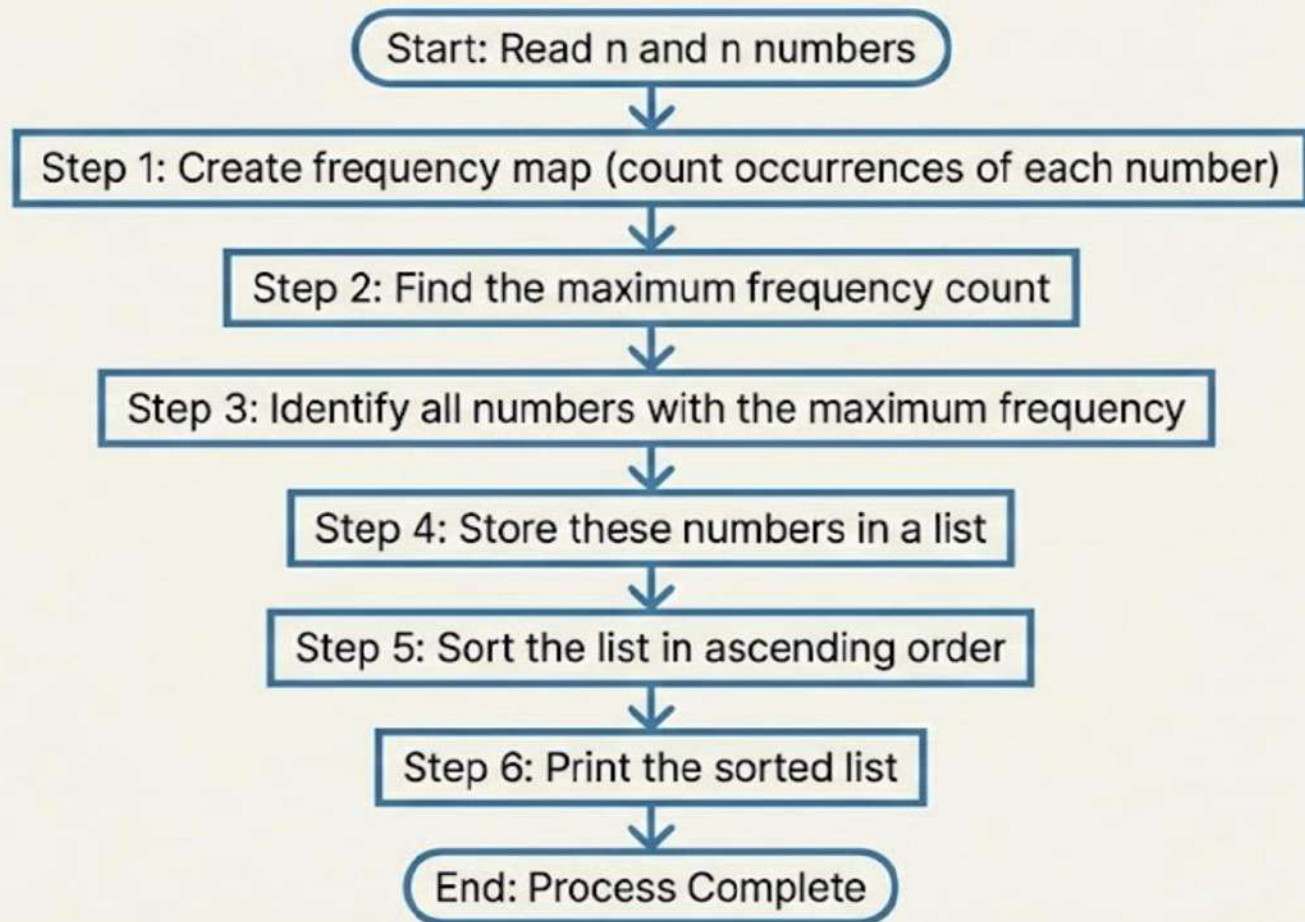
Numbers with max frequency (3): `{3, 5}`

→ Sort in ascending order:
`[3, 5]`

The most frequent numbers sorted
are 3 and 5.

Result: `[3, 5]` → Overall: `[3, 5]`

The Logic Flowchart



The Explanatory Approach

Step-by-Step implementation for clarity.

```
# Step 1: Read the input size n
n = int(input())
```

Reads the total count of input numbers.

```
# Step 2: Read all numbers into a list
numbers = []
for _ in range(n):
    numbers.append(int(input()))
```

Collects all the input integers into a single list.

```
# Step 3: Count the frequency of each number
frequency_map = {}
for num in numbers:
    if num in frequency_map:
        frequency_map[num] += 1
    else:
        frequency_map[num] = 1
```

Uses a dictionary to count the occurrences of each number.

```
# Step 4: Find the maximum frequency
max_freq = 0
for count in frequency_map.values():
    if count > max_freq:
        max_freq = count
```

Finds the highest frequency among all numbers.

```
# Step 5: Collect all numbers with the maximum frequency
result_list = []
for num, count in frequency_map.items():
    if count == max_freq:
        result_list.append(num)
```

```
# Step 6: Sort the results in ascending order
result_list.sort()
```

Sorts the final list of most frequent numbers in ascending order and prints it.

```
# Step 7: Print the final sorted list
print(result_list)
```

The Pythonic Optimization

Leveraging built-ins for concise logic.

Type Hinting
for clarity.

List Comprehension
& `Counter` object.

```
def most_frequent_sorted(numbers: list[int]) -> list[int]:  
    return sorted([num for num, count in Counter(numbers).items()  
                    if count == max(Counter(numbers).values())])
```

Single-line list
comprehension combines
counting, filtering, and sorting

Result: Identical output, cleaner syntax

Concept Spotlight: Frequency Counting with Dictionary

A simple and efficient way to count occurrences of elements in a collection using key-value pairs.

In Python, a dictionary is ideal for counting frequencies. You iterate through a list, and for each element, you either add it to the dictionary with a count of 1 (if it's new) or increment its existing count (if it's already there).

Code View

```
counts = {}  
for item in data:  
    counts[item] = counts.get(item, 0) + 1
```

Runtime Object

Input data = [3, 5, 3, 5, 3, 2]

counts = {}

→ counts = {3: 1}

→ counts = {3: 1, 5: 1}

→ counts = {3: 2, 5: 1}

→ counts = {3: 2, 5: 2}

→ counts = {3: 3, 5: 2}

→ counts = {3: 3, 5: 2, 2: 1}

3

3

5

2

2

1

Application: Count number frequencies. **Input:** [3, 5, 3, 5, 3, 2] → **Output:** {3: 3, 5: 2, 2: 1}

Concept Spotlight: Filtering and Sorting for Maximum Frequency

Identify elements with the highest occurrence count and arrange them in ascending order.

Step 1: Get Max Count

Identify the highest value in the frequency map.

Step 2: Filter for Max Frequency

Select all numbers (keys) whose count equals the maximum frequency.

Step 3: Sort and Return

Sort the filtered numbers in ascending order and return as a list.

Scenario A:

Input Frequencies: {3: 3, 5: 2, 2: 1}.
Max count is 3.
Numbers with count 3: [3].
Sorted list: [3].

Scenario B:

Input Frequencies: {3: 2, 5: 2, 8: 1}.
Max count is 2.
Numbers with count 2: [3, 5].
Sorted list: [3, 5].

Scenario C:

Input Frequencies: {5: 3, 3: 3, 8: 1}.
Max count is 3.
Numbers with count 3: [5, 3].
Sorted list: [3, 5].

Robustness Rule: When sorting, always consider whether the problem requires ascending or descending order and use the appropriate sorting method.

Solution Comparison



Step-by-Step Approach

- **Manual Counting:** Uses a dictionary and a loop to count how many times each number appears.
- **Find Max Count:** Loops through the counts to find the highest frequency.
- **Filter & Collect:** Creates a new list by selecting only the numbers that have the maximum count.
- **Sort & Print:** Sorts the final list of most frequent numbers and prints it.



Concise One-Liner

- **Automated Counting:** Uses `Counter()` to instantly get frequency counts.
- **List Comprehension Filter:** A single line that selects numbers whose count matches the maximum count.
- **Built-in Sorting:** The filtered list is passed directly to `sorted()`.
- **Benefit:** The entire process is a single, readable expression without explicit loops.

Key Takeaways

01

Problem: Most Frequent Numbers

The goal is to find the numbers that appear most often in a given input list. The final output must be a sorted list of these numbers.

02

Core Logic: Frequency Counting & Filtering

Count the occurrences of each number using a frequency map. Then, filter for all numbers whose count equals the maximum frequency.

03

Return Value: Sorted List

The function should return a single list containing all the most frequent numbers, sorted in ascending order.

Final Syntax Cheat Sheet

```
return sorted([num for num, count in Counter(numbers).items() if count == max(Counter(numbers).values())])
```